Polynomial Representation Using Linked List

To build a polynomial, we break it down into parts. Each part is a **term**. A term has a **coefficient** (the multiplier) and an **exponent** (the power).

We can store this using a **Linked List**. Each **Node** in the list will represent exactly one term. We also need a global pointer to keep track of the very first node.

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Define the Node structure
struct Node {
    int coeff;
    int exp;
    struct Node *next;
} *poly = NULL; // Global pointer for the starting node
```

Step-by-Step Logic:

- `coeff` : Stores the coefficient (e.g., the '2' in $2x^3$).
- `exp` : Stores the exponent (e.g., the '3' in $2x^3$).
- `next` : A pointer linking to the next term in the polynomial.
- `poly` : The head pointer. It starts as `NULL` because the list is initially empty.

Node Structure Space Complexity

Structure	Space Complexity	Reason
Single Node	**O(1)**	Fixed memory for two integers and one pointer.
Entire Polynomial	**O(n)**	Where *n* is the total number of terms in the polynomial.

Creating the Polynomial

We need to build the list by taking inputs from the user. We will ask how many terms there are. Then, we will use a loop to grab the coefficient and exponent for each term and attach the new node to the **end** of the list.

```
void create() {
    struct Node *t, *last = NULL;
    int num, i;

    printf("Enter number of terms: ");
    scanf("%d", &num);

    printf("Enter each term with coefficient and exponent:\n");
    for(i = 0; i < num; i++) {
        // 1. Create a new node in memory
        t = (struct Node *)malloc(sizeof(struct Node));

        // 2. Read the data
        scanf("%d %d", &t->coeff, &t->exp);
        t->next = NULL; // This will be the last node, so next is NULL

        // 3. Link the node
        if(poly == NULL) {
            // If it is the very first node
            poly = last = t;
        } else {
            // Insert at the end of the list
            last->next = t;
            last = t;
        }
    }
}
```

Step-by-Step Logic:

- We use `t` to create new nodes. We use `last` to keep track of the tail of the list.
- We run a `for` loop based on the total number of terms (`num`).
- Inside the loop, `malloc` grabs memory for a new node.
- We read the `coeff` and `exp` from the keyboard.
- If `poly` is `NULL` , the list is empty. We make `poly` and `last` point to this first node.
- If the list already has nodes, we connect the new node to `last->next` , and then update `last` to point to this new end node.

Create Operation Complexity

Complexity Type	Big O	Reason
Time Complexity	**O(n)**	We loop exactly *n* times to read *n* terms.
Space Complexity	**O(n)**	We dynamically allocate memory for *n* nodes.

Displaying the Polynomial

To see what we built, we must travel through the linked list from the first node to the last node. We will print the values in a standard math format like `2x3 +  .`

```
void display(struct Node *p) {
    while(p != NULL) {
        printf("%dx%d", p->coeff, p->exp);

        // Print a plus sign if there is a next term
        if(p->next != NULL) {
            printf(" + ");
        }

        p = p->next; // Move to the next node
    }
    printf("\n");
}
```

Step-by-Step Logic:

- We pass the starting node `p` into the function.
- The `while` loop runs until `p` becomes `NULL` (end of the list).
- We print the current node's coefficient and exponent.
- We shift `p` to `p->next` to step forward.

Display Operation Complexity

Complexity Type	Big O	Reason
Time Complexity	**O(n)**	We visit every node exactly once.
Space Complexity	**O(1)**	We only use one temporary pointer `p` .

Evaluating the Polynomial

Evaluating means we plug a specific number into `x` and calculate the final total. We will use the built-in `pow()` function from the `<math.h>` library to calculate $x^{exponent}$.

```
long Eval(struct Node *p, int x) {
    long val = 0; // Starts at 0 to accumulate the sum

    while(p != NULL) {
        // val = val + (coefficient * (x ^ exponent))
        val += p->coeff * pow(x, p->exp);
```

```
        p = p->next; // Move to the next node
    }

    return val;
}
```

Step-by-Step Logic:

- We create a `val` variable initialized to `0`. We use `long` because math results can get very large.
- We loop through the list.
- For each node, we do: `coeff * pow(x, exp)`.
- We add that result to our running total `val`.
- We move to the next node. Once finished, we return the total.

Evaluate Operation Complexity

Complexity Type	Big O	Reason
Time Complexity	**O(n)**	We visit every node exactly once to do the math.
Space Complexity	**O(1)**	We only store a single variable `val` for the total.

The Main Function & Dry Run

Finally, we put everything together in the `main` function to test our code.

```
int main() {
    create();

    printf("Polynomial is: ");
    display(poly);

    // Evaluate the polynomial with x = 1
    int x_value = 1;
    long result = Eval(poly, x_value);

    printf("Result when x=%d is: %ld\n", x_value, result);

    return 0;
}
```

Trace & Dry Run

Let's test the evaluation step by doing a manual dry run.

- **Inputs Given:** 4 terms.
- **Term 1:** Coefficient = 3, Exponent = 5 ($3x^5$)
- **Term 2:** Coefficient = 2, Exponent = 4 ($2x^4$)
- **Term 3:** Coefficient = 2, Exponent = 3 ($2x^3$)
- **Term 4:** Coefficient = 2, Exponent = 0 ($2x^0$)
- **Polynomial Built:** $3x^5 + 2x^4 + 2x^3 + 2x^0$

Evaluating with `x = 1`:

- **Start:** `val = 0`
- **Node 1:** $3 \times (1)^5 = 3$. New `val` $= 0 + 3 = 3$.
- **Node 2:** $2 \times (1)^4 = 2$. New `val` $= 3 + 2 = 5$.
- **Node 3:** $2 \times (1)^3 = 2$. New `val` $= 5 + 2 = 7$.
- **Node 4:** $2 \times (1)^0 = 2$. New `val` $= 7 + 2 = 9$.
- **Final Output:** The result is **9**.